

Two-Stage Intrusion Detection System in Intelligent Transportation Systems Using Rule Extraction Methods From Deep Neural Networks

Samah Almutlaq, Abdelouahid Derhab^{ID}, Mohammad Mehedi Hassan^{ID}, *Senior Member, IEEE*,
and Kuljeet Kaur^{ID}, *Member, IEEE*

Abstract—In recent years, intrusion detection systems (IDSs) are offering effective solutions to protect various types of cyber-attacks in different networks such as Internet of Vehicles (IoVs) network in Intelligent Transportation Systems (ITS). Deep learning models have largely been leveraged by these intrusion detection systems to achieve better effectiveness results. However, deep learning models are black boxes, which limits their acceptability in decision systems. Also, they require powerful processing capabilities such as GPU, which limit their deployments in resource-constrained devices in IoV environment. To deal with these issues, we propose a two-stage IDS in ITS to discover suspicious network activity of In-Vehicles Networks (IVN) and vehicles to everything (V2X) networks. Our proposed IDS system uses rule extraction methods from deep learning models, i.e., deep neural networks in two stages. In the first stage, we analyze network traffic to distinguish between normal and attack traffic. If the traffic is found malicious, the second stage is invoked to identify the type of attack. To this end, we propose three variants of rule extraction. The first and the second variants are homogeneous, and they apply *DeepRed* and *HypInv* rule extraction methods in both stages respectively. The third variant is heterogeneous, and it applies *HypInv* in the first stage to perform binary classification, and *DeepRed* in the second stage to perform attack classification. The key idea is to combine the advantages of rule extraction technique and two-stage IDS architecture to resource consumption and improve classification accuracy. The proposed IDS model was tested using four benchmark datasets, i.e. ISCXIDS2012, CIC-IDS2017, and CSE-CIC-IDS2018 datasets are used for external network communications and the car hacking dataset are used for in-vehicle communications. The evaluation results show that the homogeneous *DeepRed* is the optimal one in all cases of IDS system with an accuracy scores ranging between 92.43%-98.32% under CIC-IDS2017 dataset, between 91.32%-99.46% under CSE-CIC-IDS2018 dataset, and between 96.05%-99.21% under Car-hacking dataset.

Index Terms—Intelligent transport systems, Internet of Vehicles, intrusion detection system, rule extraction, *DeepRed*, *HypInv*, deep neural network.

I. INTRODUCTION

INTELLIGENT Transportation Systems (ITS), especially Autonomous Vehicles (AVs), are able to communicate with other vehicles and infrastructures through vehicle-to-everything (V2X) network, and each vehicle is also capable of performing autonomous driving tasks, such as self driving, following the lane, self parking, collision avoidance, through In-Vehicle Network (IVN) that connects different electronic control units (ECUs). The role of ECU is to receive information from sensors and send the required instructions to the actuators, which carry out the corresponding autonomous functionalities.

Autonomous Vehicles are vulnerable to safety and security issues that undermine the human lives. At the point when AVs are connected to the internet (known as the Internet of Vehicles (IoVs)), they are exposed to different types of cyber-attacks due to their inherent security vulnerabilities related to involved IoT devices and their communication technologies. Intrusion detection systems (IDSes) can play a major role to protect IoVs from multiple types of cyber attacks that aim to compromise the confidentiality, integrity, availability, and authenticity of the network, such as man-in-the-middle attack, eavesdropping attack, message tampering attack, denial of service (DoS)/ distributed denial of service (DDoS), jamming attack, spoofing attack, accessing OBD-II port of the vehicle and injecting malicious traffic into the IVN [1]. They can be deployed to identify these attacks when the preventive countermeasures fail by monitoring host and network activities [2], [3], [4].

Deep learning techniques have been widely investigated in the intrusion detection systems in ITS [4], [5], [6], [7], [8] as they are capable of dealing with large amounts of data and distinguishing between normal and attack traffic with high accuracy. However, they face two major challenges. The first challenge is the deep learning models are black boxes, and it is not easy to explain how the models work, and how they make their decision, and limits the acceptability of DNN products in decision systems [9], [10]. For instance, if the IDS fails at correctly classifying the network traffic in INV, the ECU could send incorrect instructions, such as

Manuscript received 24 May 2022; revised 2 August 2022; accepted 23 August 2022. Date of publication 8 September 2022; date of current version 29 November 2023. This work was supported by King Saud University, Riyadh, Saudi Arabia, through the Researchers Supporting Project, under Grant RSP2023R18. The Associate Editor for this article was Z. Lv. (Corresponding author: Mohammad Mehedi Hassan.)

Samah Almutlaq and Mohammad Mehedi Hassan are with the Department of Information Systems, College of Computer and Information Sciences, King Saud University, Riyadh 11543, Saudi Arabia (e-mail: samah-ksa@hotmail.com; mmhassan@ksu.edu.sa).

Abdelouahid Derhab is with the Center of Excellence in Information Assurance (CoEIA), King Saud University, Riyadh 11451, Saudi Arabia (e-mail: abderhab@ksu.edu.sa).

Kuljeet Kaur is with the Electrical Engineering Department, Ecole de technologie supérieure, Montreal, QC H3C 1K3, Canada (e-mail: kuljeet.kaur@ieee.org).

Digital Object Identifier 10.1109/TITS.2022.3202869

1558-0016 © 2022 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See <https://www.ieee.org/publications/rights/index.html> for more information.

turning off the vehicle engine, and sudden brakes, which compromise the safety of passengers, the vehicle, and the environment [11]. The second challenge is that they consume enormous resources in terms of computation and memory. They also require powerful processing capabilities such as GPU, which limit their deployment in resource-constrained devices, such as ECUs [2].

Rule-based machine learning like decision trees or simple if-then rules could address the two above-mentioned challenges, as they are easier to understand than deep neural network and provide better time and resource efficiency, and is a way to get insights from the knowledge that are embedded in the neural network in the form of rules [12]. The extraction of rules from deep neural networks has been proposed in [13] and [14] to combine the advantages of both deep learning and rule-based methods, as it extracts comprehensible rules from deep models and provides time efficiency for IDSes. It also provides an explanation ability to the deep learning through understanding the inner model architecture.

The main role of intrusion detection system is to distinguish between normal traffic and attack traffic. To this end, binary classification models could be employed. However, security analysts require further information such as the type of attack, in order to identify the required mitigation response. To fulfil this requirement, binary models are not sufficient, and multi-class classification models can be applied. By comparing between the two types of classification models, we can find that the binary models consider classification problem of two classes, which less complex than the multi-class models, and hence the binary models could provide better accuracy, especially when the number of classes in the multi-class problem is high. On the other hand, the binary models do not provide the type of attack as the multi-class ones, and hence they are not helpful for security analysts to plan a response against the attack. To deal with this issue, some IDSes have been proposed to adopt a two-stage architecture [15], [16], which combines the two models: binary and multi-class. In the first stage, a binary model is applied to distinguish between normal traffic and the attack traffic. If the traffic is found attack, it will be examined in the second stage using a multi-class model to identify the type of attack.

In the literature, different intrusion detection systems have been proposed for IoVs [2], [17], [18], [19], [20], [21]. However, the following two main research gaps can be observed:

- Rule extraction from DNN is not investigated in the context of IDS in general and ITS in particularly. More specifically, prior research works, which focused on extracting rules in the context of intrusion detection systems, only consider neural networks with one hidden layer [22], [23], [24]. There are only two works that investigated rule extraction from deep neural networks in the context of malware detection [25], [26], but no work considered intrusion detection system.
- Combination of rule extraction and two-stage IDS is not investigated. Prior works either considered Two-stage IDS or rule extraction from deep neural networks.

In this paper, we make the following contributions:

- Our work is the first that investigates rule extraction from deep neural networks in the context of IDS in Intelligent Transportation System.
- We propose an novel intrusion detection system that leverages two concepts: rule extraction from deep neural network, and two-stage IDS, and integrates them into one architecture. To the best of our knowledge, there is no previous work that considered this integration for intrusion detection systems in ITS environment. The basic idea behind this integration is to inherit the advantages of both concepts, namely (i) explainable DNN and efficiency in terms of time and resource by rule extraction, and (ii) better classification accuracy by two-stage architecture.
- We propose a novel architecture of two-stage intrusion detection system, where a rule extraction method from deep neural network is applied in each stage. Based on this architecture, we propose three variants. The first and the second variants are homogeneous and apply *DeepRed* [13] (i.e., a compositional rule extraction approach), and *HypInv* [14] (i.e., pedagogical rule extraction approach) in both stages respectively. The third variant is heterogeneous, and applies *HypInv* in the first stage and *DeepRed* in second stage.
- We employ a hybrid feature selection method that incorporates three filter-based and two wrapper-based methods. The proposed feature selection method helps eliminating unrelated features for model training improvement.
- We conduct extensive experiments, and evaluate the proposed intrusion detection system on four network intrusion detection system datasets, namely ISCX-IDS2012 [27], CIC-110 IDS2017 [28], CSE-CIC-IDS2018 [29], and Car-hacking [30].

The rest of the paper is organized as follows: Section II gives a background on rule extraction methods. Section III describes the literature review. Section IV presents the design of the proposed intrusion detection system, and its implementation is provided in Section V. Section VI presents and discusses the experimental results. Finally, Section VII concludes the paper.

II. BACKGROUND

A. Rule Extraction From Deep Learning

The objective of rule extraction from deep learning is to find association rules from a dataset. It is defined as follows: “Given a trained neural network and the data on which it was trained, it produces a description of the network’s hypothesis that is comprehensible yet closely approximates the network’s predictive behavior” [31]. There are the following two main rule extraction methods from deep neural networks:

- *Pedagogical Methods*: They use the trained model as a black box to find combinations of input values to output values. Because the input space can be very large, various heuristics are used to limit the search space and extract the rules that can provide the desired performance with the least amount of complexity. Some pedagogical rule extraction methods can be found in [14], [32], and [33].

- b *Decompositional Methods*: They analyze the internal structure of the neural network by splitting the network into neuron levels. The results from each neuron are aggregated to form rules that represent the network [34], [35].

B. DeepRed

DeepRed [13] is a compositional method for rule extraction from deep neural networks. It extracts rules from the DNN's intermediate layers. The extracted rules are merged to form the final rule. Continuous or binary attributes can be used to generate rules with varying degrees of fidelity while keeping the complexity of generated rule to a minimum.

C. HypInv

HypInv [14] is a pedagogical rule extraction method that is used for classification problems. It extracts rules as hyperplanes, and can create rules that treat neural networks like black boxes using hyperplane estimation. Unlike *DeepRed* [13], the extracted rules in *HypInv* [14] are without direct relation to neuron activation functions. The neural network's decision boundaries are thus created by hyperplane conjunction and disjunction. A piecewise process approximates the neural network's decision boundaries. The piecewise rule generation is not directly related to the training data because the goal of neural network inversion is to explain network behaviour rather than directly generate rules from data.

III. LITERATURE REVIEW

The Intrusion detection systems for IoV have attracted much attentions from researchers [2], [17], [18], [19], [20], [21]. In this section, we provide literature review with respect to two aspects: (1) *Rule extraction methods from deep neural network*, and (2) *Two-stage intrusion detection systems*.

A. Rule Extraction Methods From Deep Neural Network

The idea of rule extraction from deep neural network has not been applied in IDS domain. However, it has been proposed in general or specific domains. For instance, Chakraborty *et al.* [36] proposed extracted rules from a multi-hidden layer neural network, and used back propagation for fine tuning. They employed the decompositional approach by analyzing and extracting the knowledge from the neurons of each layer. The extracted knowledge from different layers are combined into the final ruleset of symbolic rules. The findings show that the rules are extracted with higher accuracy compared to other rule extraction algorithms such as RxREN [37], Re-RX [38], and *DeepRed* [13]. Odense *et al.* [39] presented a layer-wise knowledge extraction method, as they use M-of-N rules, which aim to reach the trade-off between accuracy and complexity. The authors showed that (a) Autoencoders using *relu* or *tanh* activation function, and (b) the softmax layer in Convolutional Neural Networks, are highly explainable by rule extraction with an accuracy of 99%. In [40], a rule extraction method from CNN is proposed, and is applied on Sentiment Analysis for textual data. The deep learning architecture includes a fully connected layer, and use the

Discretized Interpretable Multi-Layer Perceptron (DIMLP). Evaluation results show the rules reach high fidelity, and the model is more accurate compared to models based on SVMs and decision trees with an accuracy of 82%. Mahdaviyar and Ghorbani [25] proposed DeNNeS, deep embedded neural network expert system, which extracts refined rules from a trained deep neural network. DeNNeS is evaluated on two datasets: UCI phishing websites dataset, and the Android malware dataset. The extracted rules an accuracy of 97,5%, which approximates the accuracy of the neural network. DeNNeS outperforms other machine learning model such as: random forest, support vector machine, k-nearest neighbor, and Gaussian naive Bayes. Yan *et al.* [26] proposed a rule extraction method from deep neural network by constructing input-hidden tree for each hidden layer. Then, all these tress are merged into one rule, which is used to analyze network traffic, and hence detecting mobile malware.

B. Two-Stage Intrusion Detection Systems

In the literature, many IDSes [41], [42] adopted the design of two-stage architecture to detect an classify attacks. However, this design has not been found in the context of IoVs. Bovenzi *et al.* [41] proposed H2ID, a two-stage hierarchical Network Intrusion Detection system. In the first stage, anomaly detection system based MultiModal Deep AutoEncoder (M2-DAE) is applied to detect anomalous traffic. The second stage identifies the class of the anomaly using soft-output classifiers. H2ID was tested and validated using Bot-IoT dataset [42]. The same architecture is adopted in [43]. Hussain *et al.* [44] proposed a two-stage hybrid detection method, which applies anomaly detection using Support Vector Machine (SVM) in the first stage, and misuse detection using Artificial Neural Network (ANN) in the second stage to identify the attack class. Khan *et al.* [16] proposed TSDL, an intrusion detection system consisting of two stages: initial and decision, and a deep stacked auto-encoder is implemented in each stage. In the initial stage, network traffic is classified into normal or attack using the softmax classifier. In the decision stage, the probability score of the softmax function is used as an input feature to a multi-class classifier to classify the traffic as normal or any other attack class. Umer *et al.* [45] proposed a two-stage IDS that is based on network flow traffic. In the first stage, a one-class support vector machine (OC-SVM) is employed to distinguish between normal and attack flows. In the second stage, self-organizing map is applied to group similar attack flows into different attack clusters. Li *et al.* [46] proposed a two-step hybrid intrusion detection system. In the first step, different binary classifiers and one aggregation module are employed to identify the class of network connections. In the second step, the uncertain connections are analyzed by k-NN algorithm to determine their exact classes.

C. Comparison With Literature Review

Our work differs from the literature review in the following points:

- Rule extraction from DNN has not been investigated in IDS. Prior research works, which focused on extracting

rules in the context of intrusion detection systems, only consider neural networks [22], [23], [24]. This means that the network contains one hidden layer. The extraction of rules from deep neural networks has only been investigated by few works [9], [12], [36], [47] and it is still in its infancy stage, especially in cybersecurity domains. There are only two works that investigated rule extraction from deep neural networks in the context of malware detection [25], [26]. To the best of our knowledge, our work is the first that tackles rule extraction from deep neural networks with respect to intrusion detection systems in general and ITS in particularly.

- We can observe that the above described works either adopt rule extraction from DNN or the two-stage IDS design, and there is no work that has considered the combination of rule extraction from DNN and Two-stage IDS architecture. To the best of our knowledge, there is no previous work that considers this integration for intrusion detection systems in ITS environment.

IV. PROPOSED APPROACH

In this section, we describe our Two-stage intrusion detection system using rule extraction methods from deep neural networks. First, we present the used deep neural network architecture. Then, we present our generic architecture of Two-stage intrusion detection system using rule extraction methods from deep neural networks. After that, we describe the design of compositional method, i.e., *DeepRed* and, and the pedagogical method, i.e., *HypInv*. Finally, we describe the heterogeneous and homogeneous variants of the proposed intrusion detection system.

A. Deep Neural Network Architecture

DNNs are fully connected networks, and their main advantage is that they are “structure agnostic” i.e., no assumptions are made on the input, which is compatible with the format of the IDS datasets, i.e., the tabular data columns in the IDS datasets do not have structure. Instead, CNN architectures, for instance, make the explicit assumption that the inputs are images in order to allow encoding certain properties into the model architecture. Also, the image datasets are structured data, i.e., the input is matrix of pixels, with width and height. Thus, the proposed IDS extracts the rules in the form of trees, from a fully connected feed forward neural network, which is made up of (a) an input layer, (b) a series of fully connected layers, called hidden layer, and (c) an output layer. Each hidden layer receives inputs from its previous layer and sends outputs to its next layer, as shown in Figure 1. Each neuron of the hidden layer applies an activation function, as shown in Equation (1), where z , x_i , w_i , and b denote the output value, the input value, the weight, the bias values respectively.

$$z = \sum_i w_i \star x_i + b \quad (1)$$

The rules are retrieved from the neurons using the Rectified Linear (ReLU) activation function, as shown in Equation (2).

$$f(x) = \begin{cases} x & \text{if } x > 0, \\ 0.01x & \text{otherwise.} \end{cases} \quad (2)$$

ReLU can increase the learning speed of deep neural networks when the gradients of the ReLU activation function become constant and do not vanish at positive levels [48]. The softmax function is used to normalize the Deep Neural Network’s final layer, as shown in Equation (3).

$$\sigma(z)_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} \quad \text{where } K \text{ is the number of classes.} \quad (3)$$

Depending on the error rate achieved in the previous epoch, back-propagation is used to fine-tune the weights of a neural network. Because of the wide variety of rule extraction methods and the close relationship between the semantics of extracted rules and the semantics of the application domain, no rule extraction method is a standard, and it is always necessary to select a method that is appropriate for the application domain. The goal of supervised learning is to learn a function by getting the DNN output as near as possible to the intended. Consequently, calculating the difference between the DNN’s intended and actual output requires the use of a loss function. A cross-entropy loss function replaces the usual mean squared error in the classification domain.

B. Two-Stage Intrusion Detection Using Rule Extraction

1) *Approach Overview*: Figure 2 presents the general architecture of the proposed intrusion detection system, which operates in two stages. In the first stage, we extract a rule set from DNN using “Rule Extraction Method 1”, and apply binary classifier, which distinguish between two classes: normal and attack traffic. In the second stage, only attack traffic is considered. we extract a rule set from DNN using “Rule Extraction Method 2”, and apply multi-class classifier, which attributes the analyzed network traffic to its attack class.

Based on the general architecture, depicted in Figure 2, we derive in Figure 3 the following three variants:

- *Homogeneous DeepRed Variant*: The used rule extraction method in both stages is *DeepRed*
- *Homogeneous HypInv Variant*: The used rule extraction method in both stages is *HypInv*.
- *Heterogeneous Variant*: *HypInv* and *DeepRed* are used in the first and the second stages respectively.

2) *Compositional Approach: DeepRed*: Comparative analysis of rule extraction methods [49] shows that *DeepRed* algorithm is a promising de-compositional approach for MLP and DNN. This is due to the most published compositional algorithms unable to deal with neural network with more than one hidden layer. *DeepRed* was first proposed in [35], and its recent improved version can be found in [49]. The basic idea of *DeepRed* is to create decision trees (DTs) using the activation of neural network as input, and then merge these trees into Decision Directed Acyclic Graph (DDAG).

The unique splits of *DeepRed* [49] are generated for each output class in the output layer, as shown in Figure 4. In the figure, we can find the unique splits for output class o_1 , named as functions. o_1 functions, as in the figure, split data by 0.5 starting from h_3^1 and split data by 0.2 when ending with h_3^5 . Accordingly, the algorithm extracts the rule sets that describe this output neuron using the neurons’ activation

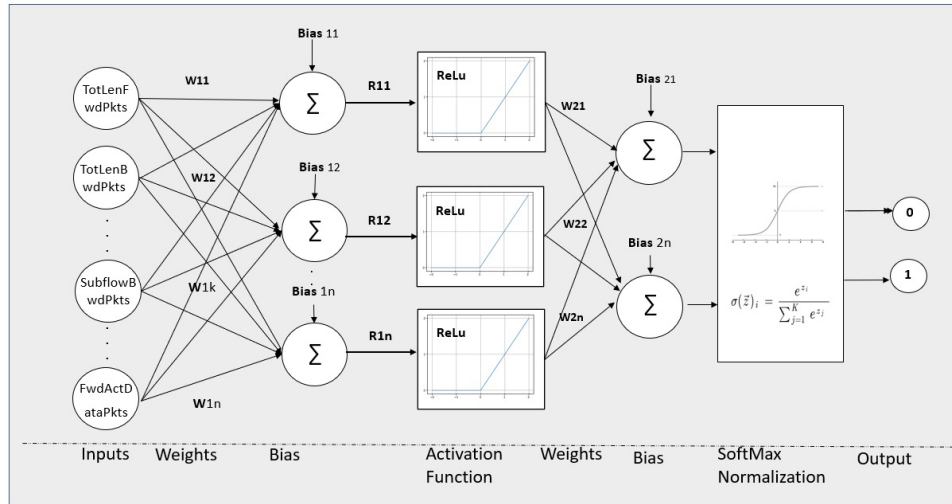


Fig. 1. Proposed Deep Neural Network architecture.

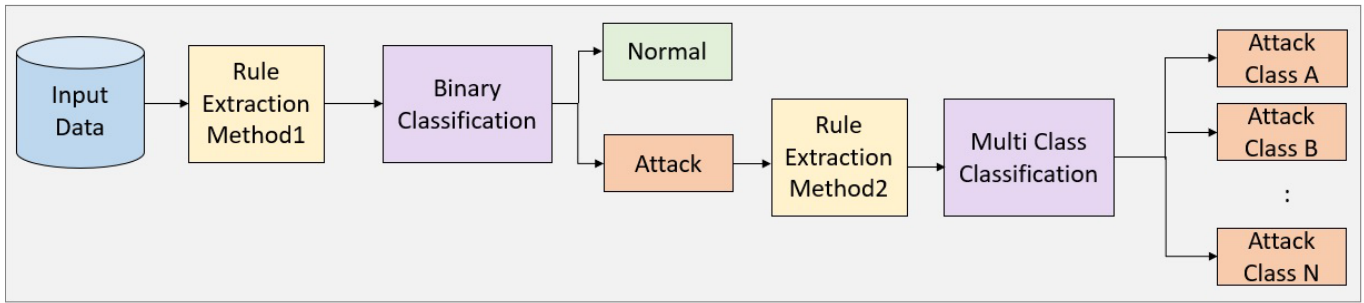


Fig. 2. Two-stage framework for network intrusion detection system.

functions of the previous layer, named Intermediate Rule. represented as red lines in the tree of Figure 4. The generated rule sets are composed of neurons' activation functions that lead to output $o1$.

For each hidden layer, the algorithm goes backwards by traversing until reaching the input layer. In this step, the algorithm takes each neuron activation function value as *Target*. For the example, the target neuron is h_3^1 in Figure 5, and after finding all its unique splits, we add them to the output class $o1$ functions. After finding all input rules for the neurons in the *Intermediate rule*; i.e., building an *InputRule* decision tree for each function, the algorithm merges together all *InputRule* for each layer, to finally substitute *Intermediate rule* terms within the *InputRule*, as shown in Figure 6. The algorithm replaces h_3^1 in the *Intermediate rule* for output $o1$ with its *InputRule*, presented in red color in the figure. Finally, it remove redundant rules using the Expansion of c/d rule merging algorithm based on J-measure, as explained in [35].

3) *Pedagogical Approach: HypInv*: *HypInv* is one of the rarest pedagogical approaches that can be applied on different neural network classifiers with graded output function. Furthermore, *HypInv* can be applied to differentiable and non-differentiable networks. The neural network inputs can be either binary or continuous, as well as binary or multi output. The original *HypInv* generates rules in the form of

hyperplane conjunction and disjunction [14]. Since *HypInv* belongs to the pedagogical approach, it does not analyze the neural network architecture. Instead, it treats the neural network as a black box, and the extracted rules have nothing to do with the neural network's weights. *HypInv* finds the network decision boundaries as conjunction and disjunction of hyperplanes such as the TanHyp in Figure 7, i.e. it approximates the neural network decision boundary in a piecewise manner.

In our intrusion detection system, we adopt an extended version of *HypInv* [49], as it enables *HypInv* to extract rules from deeper neural networks. The aim of *HypInv* algorithm is to find hyperplanes, which are tangent to the hypersurface of the decision boundary of neural network [14]. Furthermore, an input space is divided into two halves: a positive half-space S^+ , and a negative half-space S^- . The hyperplane decides the data points belonging to the positive or negative half-space. Then, the decision boundary of the network allows generating rules, which are formed of conjunctions and disjunctions of diverse rules denoted by A_i . The rule is a set of linear inequalities that form positive half-spaces that are bordered on one side by a hyperplane [14]. Let R^+ be the volume created by the conjunction and disjunction of those half-spaces, where the rule base classifies the input patterns as belonging to the C^+ subclass. Let $R^- = \neg R^+$, and $R^- \vee R^+ = S$. R^+ is an approximation of S^+ . For example:

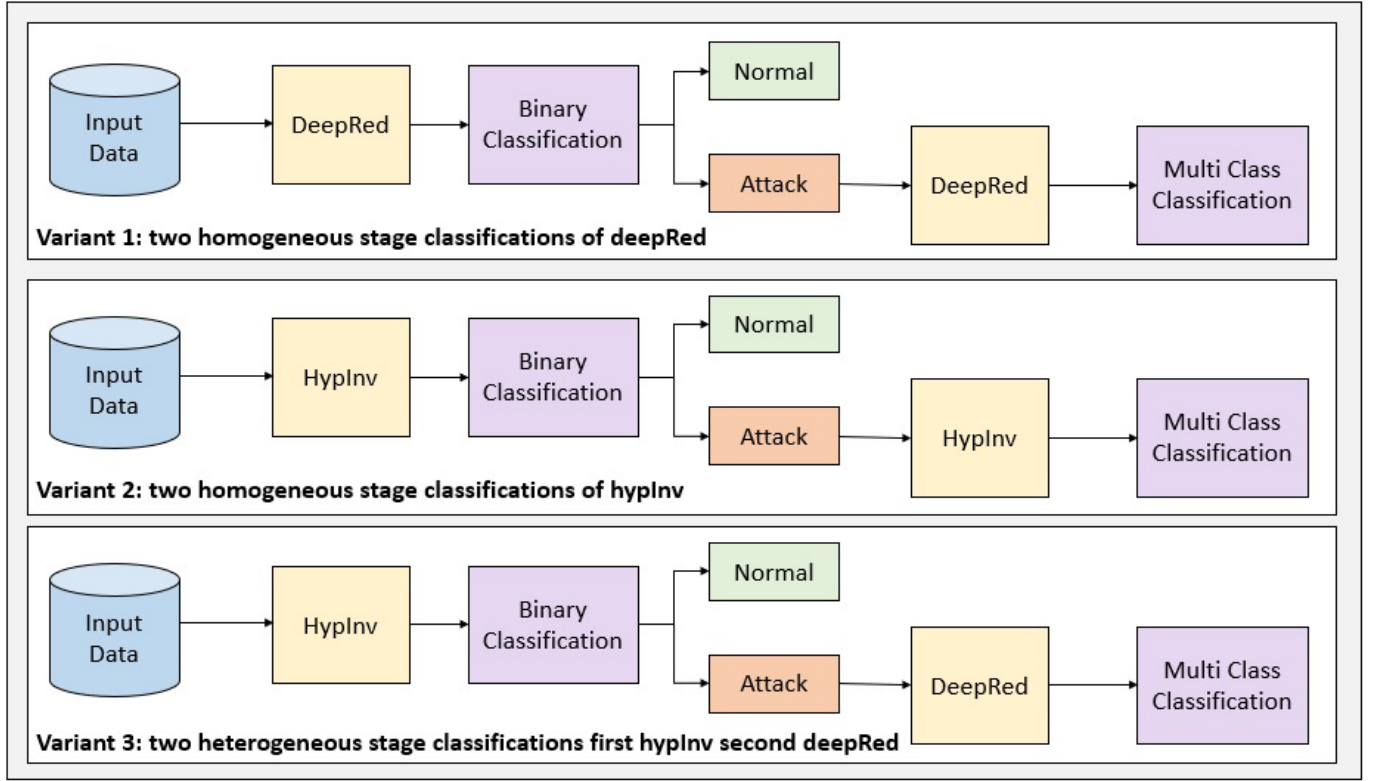


Fig. 3. Two stage homogeneous and heterogeneous variants.

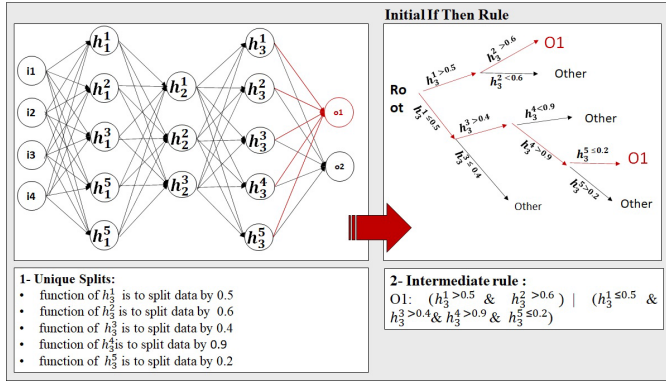


Fig. 4. An illustrative example of extracting unique splits or functions, intermediate rules.

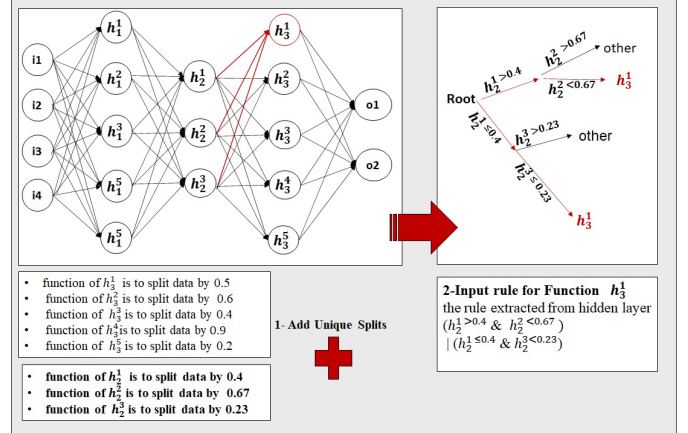


Fig. 5. Example of extracting unique splits or functions, intermediate rules.

If $A_1 \vee A_2 \wedge \vee A_3 \vee A_4$ then $x \in C+$, where

$$A_1 : 0.5 x_1 - x_2 + \dots + 0.6 x_{N+1} > 0$$

...

$$A_4 : 3 x_1 - 8 x_2 + \dots + 3 x_{N-8} < 0$$

The algorithm can generate a decision tree with oblique splits and carrying out two main steps: (a) finding the closest point, and (b) finding the next point. To find the closest point, it firstly initializes a point x_1 as a random vector in S , as shown in Figure 7. Secondly, the algorithm finds the projection of x_1 on the network decision boundary, denoted by x_0 . Thirdly, it calculates the *vector* = $x_1 x_0$, normal to the decision boundary at x_0 . Fourthly, it calculates the equation of the hyperplane tangent to the decision boundary

at x_0 : $TanHyp = n \cdot x_0$, to finally add the hyperplane split to the set of possible splits related to the created decision tree.

Before the second main step, it is worth mentioning that the original *HypInv* [14] used inversion to find x_0 , i.e., the projection of x_1 on the network decision boundary in order to calculate the first main step, i.e., the closest point to the decision boundary. The error signals are returned to the input layer by the inversion algorithm, which updates the activation values of the input units to minimize the output error. The network weights are obviously frozen during the network inversion process. We adopt the improved version of *HypInv* [49], as the modified cost function is more accurate

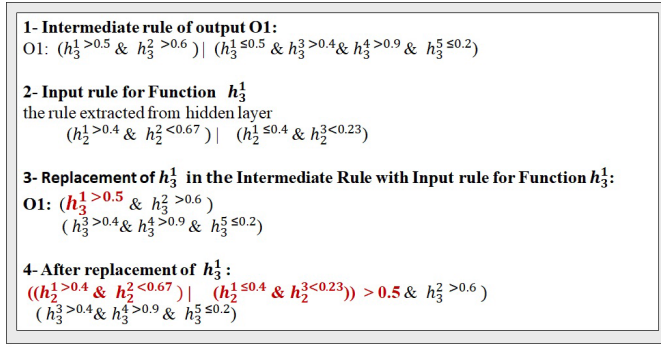


Fig. 6. Example of Substitution input rule for one neuron with intermediate rules.

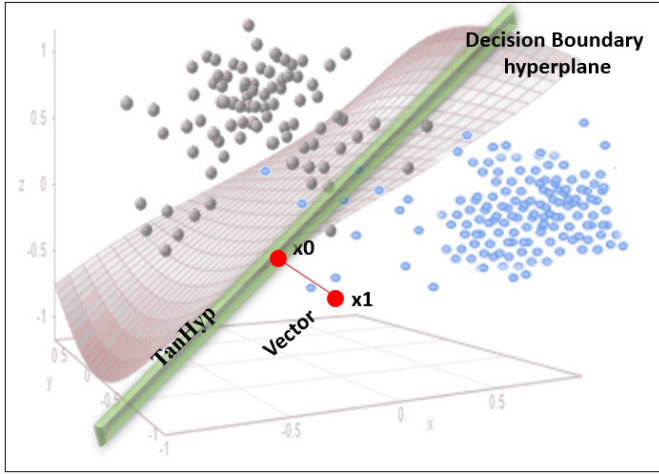


Fig. 7. Example of HyperPlane splitting.

in finding the projection of x_1 than the inversion method introduced by *HypInv* [14]. The modified cost function adds a distance between points [49], as shown in Equation (4).

$$E_d(x) = \frac{1}{2} \|t - f(x)\|^2 + \mu \|x_1 - x\|^2 \quad (4)$$

where μ is a weighting factor, and t is the desired target for sample x . This technique allows to make x close to the border by the first term and close to x_1 by the second term at the same time.

As for the second main step, we get the next target point of *HypInv* by adopting the method in [49] to find the “farthest point from a set of samples attached to the leaf with the largest error”. Theoretically, the loop of the main two steps is supposed to continue splitting hyper-planes and gets new farthest points until fidelity be more than the threshold, but practically, the number of loop cycles is restricted to a maximum limit. Thus, we adopt another major improvement of *HypInv* as in [49], since it is possible to classify more than two output classes, contrary to the original *HypInv* method [14], which cannot deal with more than two classes. To do so, *HypInv* [49] defines new function f_c instead of decision boundary, as in Equation (5).

$$f_c = \frac{f_{new}}{f_{new}^1 + f_{new}^2} \quad (5)$$

Assuming that:

$$\text{point } X_0 \in \mathbb{R}^n \text{ from class } c \in \mathbb{C} \quad (6)$$

$$f_{new}^1(x) = f^c(x), \quad f_{new}^2(x) = \max_{i \in \mathbb{C}} (f(x)^i) \quad (7)$$

The algorithm looks for a decision boundary between the class c and all the other classes, because the change of the classes happens on this boundary first.

4) *Two-Stage Homogeneous Classifier*: In the homogeneous two-stage IDS, the same rule extraction method is applied in the two stages. Thus, we have two homogeneous variants: Two-stage Homogeneous *DeepRed*, and Two-stage Homogeneous *HypInv*. In each homogeneous variant, *DeepRed* or *HypInv* is considered for both binary and multi-class classification, as shown in Algorithm 1.

Algorithm 1 Function Two Homogeneous Stages (approach, Samples)

Input: Approach, Sample X
Output: Classification C

```

1: if (approach == HypInv) then
2:    $c1 \leftarrow \text{Binary\_HypInv\_DT}(X)$   $\triangleright$  Classification
    of the sample by the first stage using HypInv binary class
    classification
3:   if ( $c1$  is preferred class) then
4:      $C \leftarrow \text{'BENIGN'}$ 
5:   else  $\triangleright$  classification of the sample by the second stage
    using HypInv Multi class classification
6:      $C \leftarrow \text{Multi\_HypInv\_DT}(X)$ 
7:   end if
8: else if (approach == DeepRed) then
9:    $c1 \leftarrow \text{Binary\_DeepRed\_DT}(X)$   $\triangleright$  Classification of
    the sample by the first stage using DeepRed binary class
    classification
10:  if ( $c1$  is preferred class) then
11:     $C \leftarrow \text{'BENIGN'}$ 
12:  else:  $\triangleright$  classification of the sample by the second stage
    using DeepRed Multi class classification
13:     $C \leftarrow \text{Multi\_DeepRed\_DT}(X)$ 
14:  end if
15: end if

```

5) *Two-Stage Heterogeneous Classifier*: In the heterogeneous two-stage IDS, *HypInv* and binary classification are applied in the first stage. As for the second stage, *DeepRed* and multi-class classification are applied, as shown in Algorithm 2. Starting from line 3 of the algorithm, *HypInv* initializes an empty *set* as an input space, then sets x_1 vector to random coordinates from *set*, then a loop starts at line 5 and stops when θ threshold is larger than the Fidelity of the decision tree on output data. To generate x_0 , i.e., the closest point to x_1 on the decision boundary, *HypInv* finds x_0 projection on the decision boundary using inversion technique at line 6. It calculates the vector that connect x_1 to its projection $x_0 = x_1 - x_0$. Geometrically, it is orthogonal (Normal) to the decision boundary at x_0 . After that, at line 7, it calculates the Orthogonal (Normal) to the decision boundary at x_0 , namely

Vector, and Calculates the equation of the hyperplane tangent to the decision boundary at x_0 , $TanHyp = Vector \cdot x_0$. Finally, if it is from the target space, it will be added to the input space set, and added as a rule set *HypInv_DT*, then continues for the next target points. In the second stage, the multi-class classifier *DeepRed* creates *IntermediateRule* in line 18. The rules are extracted from the output layer containing the rule sets. The loop in line 19 starts from layer $i - 1$ before output DNN layer until the input layer. For each neuron n in each layer *lyr*, it extracts forwarded terms from layer $lyr - 1$ to *lyr*. Furthermore, it assigns neuron n in layer *lyr* to target, as in line 21, and in line 23 there is a loop for each Unique Splits, to build an input hidden tree and combine it with the *lyr* hidden rule set. Subsequently, *LayerRuleset* in line 25 has all input Rule sets per layer. Finally, from line 28 to line 31, it merges all the rules, which are extracted from all DNN hidden layers.

V. IMPLEMENTATION

This section describe the details of the datasets followed by pre-processing, feature selection and model building stages.

A. Datasets and Pre-Processing

We conduct our experiments on two categories of datasets. The first category are used for external network communications between the vehicle and external entities such as, vehicles and infrastructures (i.e., vehicle-to-everything network), and the second category is used for In-vehicle network. Below is the description of the used datasets:

- ISCXIDS2012 [27]: It has binary labels “normal” or “attack” and it is heavily skewed class distribution.
- CIC-IDS2017 [28]: It is proposed as an improvement to ISCXIDS2012 with inclusion of HTTPs protocol, which accounts for almost 70% of network traffic. The improved dataset is multi-class dataset compared to ISCXIDS2012, and has a better class distribution.
- CSE-CIC-IDS2018 [29]: It is third release in this series, which is also a multi-class dataset. CIC-IDS2017 and CSE-CIC-IDS2018 contain fifteen categories of network traffic, out of which one is labeled “Benign” and the rest are labeled as attack type and attack sub-type.
- Car-Hacking [7]: It is dataset for In-vehicle network, which is obtained by monitoring the traffic of the in-vehicle network traffic from a genuine vehicle, and malicious traffic is injected through the OBD-II port of the vehicle. CAR HACKING dataset includes four attack categories: DoS attack, fuzzy attack, spoofing of drive gear attack, and spoofing of RPM gauge attack.

The classes in the datasets are not evenly distributed, as shown in Table I. In case of ISCXIDS2012 dataset, 97.20% of the samples are classified as “normal” and only 2.8% are classified as “attack”. Similarly, in case of CIC-IDS2017 and CSE-CIC-IDS2018 datasets 80.30% and 83% of the samples are classified as “normal” respectively, and the remaining samples are divided into six categories for CIC-IDS2017 and five categories for CSE-CIC-IDS2018. In case of CAR-HACKING dataset, 86.6% of the samples are classified as

Algorithm 2 Function Two Heterogeneous Stages (Stage, IDS_Data)

Input: Stage, IDS_Data
Output: RuleSet

```

1: if (Stage == 1) then
2:   approach=HypInv
3:   Set ← Create_MultiD(Empty)
4:   x1 ← Get_Random_Target_Point(inputSpace)
5:   while (Fidelity < 0) do
6:     x0 ← find_projection_point(IDS_Model, x1)
7:     Vector ← x1 - x0
8:     TanHyp ← Vector * x0
9:     Set ← Set ∪ (Vector * x > TanHyp)
10:    HypInv_DT ← Build_DT(Set, IDS_Data)
11:    x1 ← Get_Next_Target_point(HypInv_DT,
        IDS_Data)
12:   end while
13:   Return HypInv_DT
14: else if (Stage==2) then
15:   approach=DeepRed
16:   i=TotalNumberOfModelLayers-1
17:   while (Class_Num! =0) do
18:     IntermediateRule←Build_DT(OutputClass_num,
        hiddenLayeri(x))
19:     loop (lyr=i-1, lyr=i-2, ..., lyr=1)
20:       loop (n=1,...,until last neuron in lyr)
21:         Target ← (hiddenLayerlyrn)
22:         Functions ← AllUniqueSplits(Target)
23:         loop( for all Functions)
24:           InputRule← Build_DT(hiddenLayerlyr)split
25:           LayerRuleset← Replace(InputRule, Inter-
        mediateRule)
26:         end loop
27:       end loop
28:       loop Foreach (q=i,...,1)
29:         DeepRed_DT ← Merge (LayerRuleset)q
30:         Simplify (DeepRed_DT)
31:       end loop
32:     end loop
33:     Class_Num-1
34:     Return (DeepRed_DT)
35:   end while
36: end if

```

TABLE I
DATASETS FOR NETWORK INTRUSION DETECTION

Dataset	No. of Instances	Attack Class	Label Type
ISCXIDS2012	2,450,324	2.8%	Binary
CIC-IDS2017	2,830,743	19.7%	Multi-class
CSE-CIC-IDS2018	16,233,002	17.0%	Multi-class
CAR-HACKING	17,357,681	13.4%	Multi-class

“normal”, and the remaining samples are divided into four attack categories.

The target variable is usually the class label of the data that is assigned in an accurate and predefined mechanism.

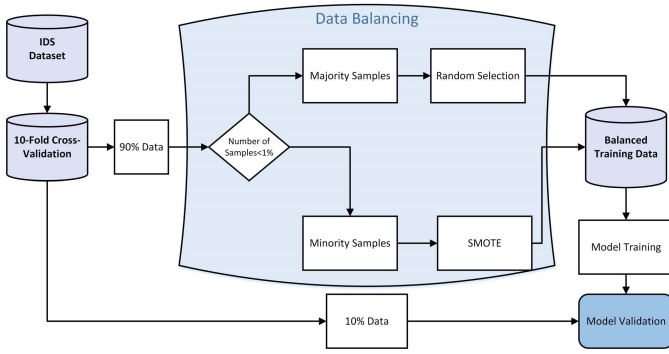


Fig. 8. Dataset balancing flow chart.

Also, the trained model has perform an accurate identification of the type of attack instead of just labelling it as “attack”. The proposed work opts for hierarchical classification, which performs the classification of an instance in two stages. The first stage performs binary classification of an instance into either “normal” or “attack”. The second stage classifies the type of attack and, therefore presenting it as a hierarchical multi-class classification problem. The categorization of an instance as “attack” and then performing the classification of type of attack requires the processing of the labels, except for ISCXIDS2012, which only contains binary labels, and therefore only the first stage is applied in case of this dataset. Normalization or scaling of the data is performed by converting the range of data into a fixed scale to ensure consistency. The network performance will improve as a result of this technique. In case of deep learning, the training data must be a matrix in the image format, and the input data must be normalized using the following equation:

$$d' = \frac{d - \min(d)}{\max(d) - \min(d)} \quad (8)$$

B. Dataset Balancing

It is clear that there is a wide variation in the distribution of samples across each of the traffic categories of the datasets, resulting in a highly imbalanced datasets. In the context of training a classification algorithm, this issue will result in biased trained model. In addition, the class with a very small number of samples will be difficult to detect among the large number of normal samples.

In order to address this issue, a simple but effective strategy is implemented. Because the samples in the majority class are extremely large and redundant, they are not used in the classifier’s training. This redundancy in the majority class will be introduced during training, and it may have a negative impact on the performance of the classification algorithm. The strategy is to compress the majority class (undersample) by reducing the number of samples to an acceptable level, and then perform expansion (oversample) of the minority classes in order to balance the number of samples in the training data. The strategy, as depicted in Figure 8, is used for data balancing in all four data sets.

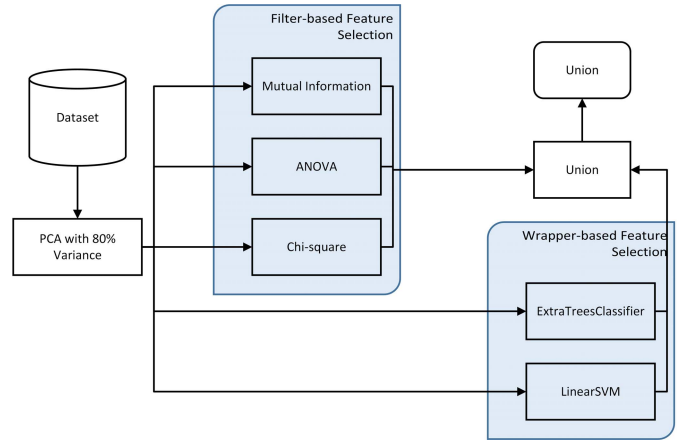


Fig. 9. Feature selection flowchart.

C. Feature Selection

Feature selection is a technique for determining which features will contribute the most to the target’s prediction. There are two fundamental approaches to perform this task: (i) predictor-specific methods (wrapper-based methods); and (ii) predictor-independent methods (filter based methods). The third approach for feature selection is to adopt a hybrid strategy. It is not recommended to use a wrapper-based method to address the problem of multi-predictor modelling because it will be ineffective. Wrapper-based methods, on the other hand, are more accurate in terms of selecting appropriate features because they are specific to a predictor. As a result, a hybrid strategy is used, in which features are selected using both approaches and then combined using the union operation. Figure 9 illustrates the used feature selection methodology.

1) *Mutual Information* [50]: There are several types of entropy-based measures, and the one in [50] selects the features based on their ability to explain the target variable accurately or how much information about the target variable can be obtained using a specific feature.

2) *ANOVA*: It is a filter-based method that performs feature selection based on the statistics of the features in the set of features. The features are chosen based on their ability to maximize the separation of their mean and minimize their spread (compactness), which is the same measure that is used for the construction of an Iterative Dichotomiser 3 (ID3) algorithm [51] based decision tree in order to determine the node and split threshold values.

3) *Chi-Square*: The chi-square test is used to determine whether or not there is a statistically significant difference between the expected and observed numbers. The null hypothesis assumes that the two variables are not related, and the test returns a value based on a 95% confidence interval for the null hypothesis. Alternatively, if the hypothesis is false, the feature is selected because it indicates that the feature is dependent on the target; however, if the hypothesis is true, the feature is discarded because it indicates that the feature is not dependent on the target.

4) *LinearSVM*: It is a wrapper-based method that selects features using linear support vector machines as a basis for

TABLE II
THE NOTATION OF DIFFERENT TRAINED VARIANTS

Model Notation	Description
DNN 2012	deep neural network trained on ISCXIDS2012 dataset
DNN 2017	deep neural network trained on CIC-IDS2017
DNN 2018	deep neural network trained on CSE-CIC-IDS2018
DNN CAR-HACKING	deep neural network trained on CAR-HACKING
DeepRed 2012	DeepRed rule extraction model trained on ISCXIDS2012
DeepRed 2017	DeepRed rule extraction model trained on CIC-IDS2017
DeepRed 2018	DeepRed rule extraction model trained on CSE-CIC-IDS2018
DeepRed CAR-HACKING	DeepRed rule extraction model trained on CAR-HACKING data set
DeepRed Binary	One stage of binary class classification using DeepRed rule extraction model
DeepRed Multi	One stage of Multi class classification using DeepRed rule extraction model
Two-Stage homogeneous DeepRed	Two stages of DeepRed classifiers; where first stage is binary DeepRed classifier and the second stage is multi class DeepRed classifier
HypInv 2012	HypInv rule extraction model trained on CSE-CIC-IDS2018
HypInv 2017	HypInv rule extraction model trained on trained on CSE-CIC-IDS2018
HypInv 2018	HypInv rule extraction model trained on trained on CSE-CIC-IDS2018
HypInv CAR-HACKING	HypInv rule extraction model trained on trained on CAR-HACKING data set
HypInv Binary	One stage of binary class classification using HypInv rule extraction model
HypInv Multi	One stage of Multi class classification using HypInv rule extraction model
Two-Stage homogeneous HypInv	Two stages of HypInv classifiers where first stage is binary HypInv classifier and the second stage is multi class HypInv classifier
Heterogenous Model (HypInv & DeepRed)	Two stages of classifiers; where first stage is binary HypInv classifier and the second stage is multi class DeepRed classifier

TABLE III
COMPARISON BETWEEN DIFFERENT BINARY-CLASS CLASSIFIERS IN TERMS OF EXECUTION TIME, CPU, AND MEMORY

Average Execution Time	DNN 2012	0.52619	DNN 2017	0.9529	DNN 2018	0.4956	DNN CAR-HACKING	0.21669
	HypInv 2012	0.31411	HypInv 2017	0.8036	HypInv 2018	0.4516	HypInv CAR-HACKING	0.28102
	DeepRed 2012	0.241	DeepRed 2017	0.3764	DeepRed 2018	0.2644	DeepRed CAR-HACKING	0.21832
Average CPU Usage	DNN 2012	0.34038	DNN 2017	0.3908	DNN 2018	0.3854	DNN CAR-HACKING	0.2877
	HypInv 2012	0.33657	HypInv 2017	0.3331	HypInv 2018	0.3404	HypInv CAR-HACKING	0.28856
	DeepRed 2012	0.31456	DeepRed 2017	0.3832	DeepRed 2018	0.3146	DeepRed CAR-HACKING	0.30763
Average Memory Usage	DNN 2012	0.59169	DNN 2017	0.9158	DNN 2018	0.9891	DNN CAR-HACKING	0.13771
	HypInv 2012	0.39255	HypInv 2017	0.6125	HypInv 2018	0.6853	HypInv CAR-HACKING	0.19068
	DeepRed 2012	0.00019	DeepRed 2017	0.3114	DeepRed 2018	0.3791	DeepRed CAR-HACKING	0.15135

the selection process. By providing the features and the targets, the model is trained with L1 as a penalty and 0.01 as a box constraint, with the penalty being L1.

5) *Extra Trees Classifier*: To train the model using features and targets, decision trees is a wrapper-based feature selection method that uses Extra Trees Classifier with 40 estimators. The selection process is carried out with the help of the contributing characteristics.

VI. EVALUATION

We conduct our experiments using Google Cloud AI platform [52]. As for software, we use Python programming language, as well as several libraries, such as scikit-learn and Tensorflow. We tune the hyper parameters with 10-fold cross-validation. We also apply different sets of parameters for each round using grid search optimization algorithm. We train the DNN with 10 epochs, 1024 batch size, and three hidden layers on all the four datasets to compare its memory consumption, CPU, and execution time under two rule extraction methods *DeepRed*, and *HypInv*, which result in training six variants: (1) DNN Binary-class variant, (2) DNN Multi-class variant,

(3) *DeepRed* Binary-class variant, (4) *DeepRed* Multi-class variant, (5) *HypInv* Binary-class variant, and (6) *HypInv* Multi-class variant. All the above-mentioned models/variants are described in Table II.

WE evaluate the performance of the variants using four metrics: Accuracy, Precision, Recall, and F1 Score by calculating the results of the confusion matrix, i.e., TP (True positive), FP (False positive), TN (True negative), and FN (False negative) on basis of 10-fold cross-validation where we get 10 results against each experiment. After that, we compute the average of all 10 results.

Hence, these the six different variants are trained on all the four datasets, which in turn lead to compare between 18 different experiments in terms of execution time, CPU usage, and Memory usage. Table III and Figure 10 consider a comparison between different Binary-class classifiers in terms of execution time, CPU usage, and Memory usage. It is to be noted that the minimum execution time is given by *DeepRed* variant i.e., rule extracted using *DeepRed* for all the four datasets ISCXIDS2012, CIC-IDS2017, CSE-CIC-IDS2018, and CAR-HACKING while DNN incur the maximum time. Similarly,

TABLE IV
COMPARISON BETWEEN DIFFERENT MULTI CLASS CLASSIFIERS IN TERMS OF TIME, CPU, AND MEMORY

Average Execution Time	DNN 2017	0.96563	DNN 2018	0.78235	DNN CAR-HACKING	0.00662
	HypInv 2017	0.34409	HypInv 2018	0.37729	HypInv CAR-HACKING	0.23001
	DeepRed 2017	0.24103	DeepRed2018	0.3234	DeepRed CAR-HACKING	0.27313
Average CPU Usage	DNN 2017	0.79196	DNN 2018	0.89598	DNN CAR-HACKING	0.17694
	HypInv 2017	0.29786	HypInv2018	0.31287	HypInv CAR-HACKING	0.23056
	DeepRed 2017	0.33619	DeepRed2018	0.53593	DeepRed CAR-HACKING	0.30295
Average Memory Usage	DNN 2017	0.83442	DNN 2018	0.99121	DNN CAR-HACKING	0.10311
	HypInv 2017	0.29727	HypInv 2018	0.45475	HypInv CAR-HACKING	0.15856
	DeepRed 2017	0.29198	DeepRed2018	0.37228	DeepRed CAR-HACKING	0.02027

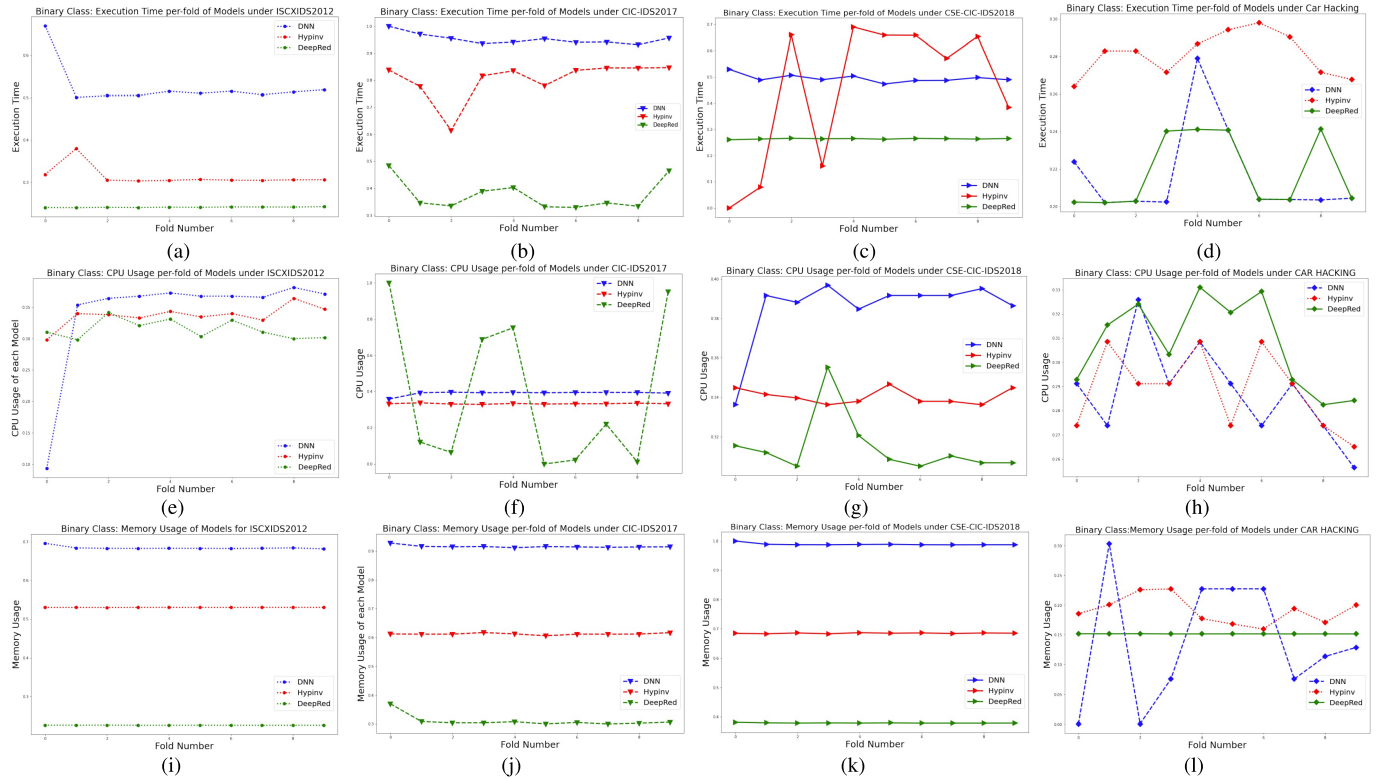


Fig. 10. Comparison between different Binary Class classifiers experiments in terms of Time (in minutes), CPU, and Memory.

DeepRed variant consumes the minimum CPU for all the four datasets and the DNN also incurs the maximum result. Even for Memory usage, *DeepRed* is the less memory-consumer, then comes *HypInv*, and DNN is the most memory-consumer. Similar trend is shown in Table IV and Figure 11 that depict the experiment results of different multi-class classifiers in terms of execution time, CPU usage, and Memory usage. *HypInv* is less performing than *DeepRed* in terms of execution time, CPU, and Memory because *HypInv* applies an iterative gradient descent optimization algorithm, which incurs additional time in finding x_0 for every hyperplane, as well as this optimization algorithm requires the availability of the whole training dataset in memory.

The classification performance of the models on the four datasets is provided in Table V. The performances of *DeepRed* is better in Car-hacking dataset because the attack patterns in Car-hacking dataset are more regular and can be better expressed as rules [11]. Thus, if we choose CAR-HACKING as reference dataset for evaluation of the different variants, we can claim that binary *HypInv* variant

outperforms *HypInv* multi-class variant. This is due to simplicity of the binary classification condition that states that each point at the boundary is defined as follows $\{x \in R^n | f(x) = (0.5; 0.5)\}$, where R^n is the input space, but for multiclass classification decision boundary is made among class c and all the other classes. Therefore, we choose the binary *HypInv* variant for the first stage classifier, and *DeepRed* multi-class variant for the second stage, as it has better F1, Accuracy, RECALL, and PRECISION in general. These two stages of different rule extraction methods are named two-stage heterogeneous classifiers /variant. Furthermore, We can observe the superiority of *DeepRed* classifier for both binary and multi-class variants on different datasets, which leads us to train another two-stage variant consisting of *DeepRed* binary variant in the first stage and *DeepRed* multi-class variant in the second stage, called two stages of homogeneous *DeepRed* classifiers. Two-stage homogeneous *HypInv* variant is also considered in the experiments. In the paper, the terms classifier and model are used interchangeably to refer to variant. As ISCXIDS2012 is a binary IDS dataset,

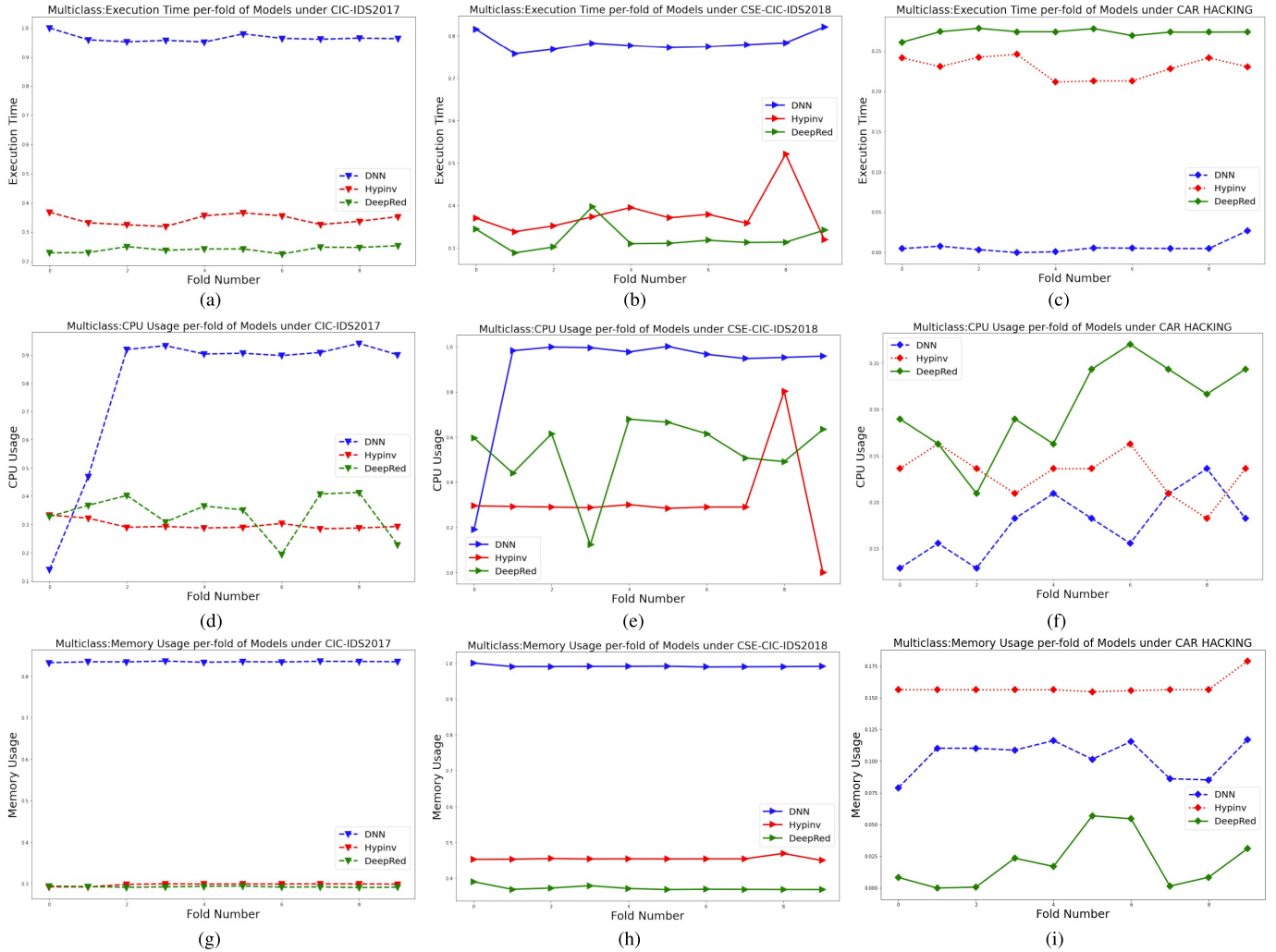


Fig. 11. Comparison between different Multi-Class classifiers in terms of Execution Time (in minutes), CPU, and Memory.

TABLE V

BINARY AND MULTI-CLASS PERFORMANCE COMPARISON OF DIFFERENT MODELS REPRESENTED AS [MIN-MAX] FOR 10-FOLD CROSS VALIDATION

Model	Dataset	F1	Accuracy	Recall	Precision
HypInv Binary	ISCXIDS2012	[68.43%-96.13%]	[59.41%-96.3%]	[85.42%-97.31%]	[53.23%-98.14%]
HypInv Binary	CIC-IDS2017	[68.16%-89.37%]	[57.47%-88.58%]	[55.46%-94.35%]	[54.27%-95.58%]
HypInv Binary	CSE-CIC-IDS2018	[79.13%-96.49%]	[79.48%-95.37%]	[86.37%-99.38%]	[73.27%-97.11%]
HypInv Binary	CAR-HACKING	[84.17%-98.23%]	[89.38%-99.48%]	[87.22%-98.24%]	[91.39%-98.31%]
HypInv Multi	CIC-IDS2017	[85.33%-95.37%]	[77.49%-93.46%]	[87.46%-96.51%]	[82.36%-95.36%]
HypInv Multi	CSE-CIC-IDS2018	[80.37%-93.45%]	[82.36%-93.26%]	[71.44%-99.21%]	[83.32%-95.36%]
HypInv Multi	CAR-HACKING	[92.13%-99.27%]	[90.43%-99.1%]	[85.34%-99.23%]	[90.3%-99.34%]
DeepRed Binary	ISCXIDS2012	[87.19%-95.15%]	[95.4%-97.34%]	[94.37%-97.28%]	[95.34%-97.16%]
DeepRed Binary	CIC-IDS2017	[95.18%-97.37%]	[95.15%-98.19%]	[95.26%-98.18%]	[93.19%-97.35%]
DeepRed Binary	CSE-CIC-IDS2018	[95.4%-99.16%]	[91.22%-96.45%]	[97.21%-99.34%]	[94.46%-99.24%]
DeepRed Binary	CAR-HACKING	[96.17%-99.47%]	[95.11%-98.34%]	[98.33%-99.35%]	[96.28%-99.27%]
DeepRed Multi	CIC-IDS2017	[90.34%-95.45%]	[81.13%-96.56%]	[92.24%-96.13%]	[86.67%-96.22%]
DeepRed Multi	CSE-CIC-IDS2018	[73.45%-98.23%]	[64.79%-98.47%]	[95.25%-99.28%]	[58.36%-98.44%]
DeepRed Multi	CAR-HACKING	[90.17%-97.15%]	[91.18%-98.38%]	[93.33%-99.36%]	[93.27%-98.36%]

we cannot use it for the Two-Stage variants either homogeneous or heterogeneous.

Thus, Figure 12 provides a bar chart of multi-class and two-stage (hierarchical) classification. The results indicate that two-stage classification approach is better as compared to multi-class classification for all scenarios. Similarly, it is

apparent that two stages of *DeepRed* is superior to two stages of *HypInv* in terms of classification accuracy. We can find that Two homogeneous stages of *DeepRed* have provided best classification accuracy of 98% and 97% for CIC-IDS2017 and CSE-CIC-IDS2018 datasets respectively. The comparison between heterogeneous and homogeneous two-stage models

TABLE VI

COMPARISON OF TWO STAGE MODELS BASED ON CLASSIFICATION PERFORMANCE REPRESENTED AS [MIN-MAX] FOR 10-FOLD CROSS VALIDATION

Model	Dataset	F1	Accuracy	Recall	Precision
Heterogenous Model (HypInv & DeepRed)	CIC-IDS2017	[91.13%-98.33%]	[92.1%-97.4%]	[95.33%-99.23%]	[91.47%-98.28%]
Heterogenous Model (HypInv & DeepRed)	CSE-CIC-IDS2018	[90.11%-98.03%]	[93.19%-98.01%]	[94.18%-99.1%]	[92.34%-98.3%]
Heterogenous Model (HypInv & DeepRed)	CAR-HACKING	[92.18%-98.15%]	[95.43%-98.12%]	[95.45%-98.23%]	[94.19%-99.22%]
Homogenous HypInv	CIC-IDS2017	[90.04%-97.43%]	[89.11%-95.34%]	[92.18%-96.11%]	[93.06%-96.15%]
Homogenous HypInv	CSE-CIC-IDS2018	[89.26%-94.46%]	[90.48%-94.35%]	[91.13%-97.3%]	[92.32%-95.49%]
Homogenous HypInv	CAR-HACKING	[92.12%-96.49%]	[91.03%-95.44%]	[92.34%-96.54%]	[94.32%-99.08%]
Homogenous DeepRed	CIC-IDS2017	[92.2%-96.32%]	[92.43%-98.32%]	[91.42%-97.46%]	[89.42%-98.32%]
Homogenous DeepRed	CSE-CIC-IDS2018	[88.1%-97.02%]	[91.32%-99.46%]	[94.32%-99.11%]	[93.36%-99.11%]
Homogenous DeepRed	CAR-HACKING	[96.05%-99.21%]	[97.43%-99.48%]	[97.32%-99.21%]	[97.44%-99.06%]

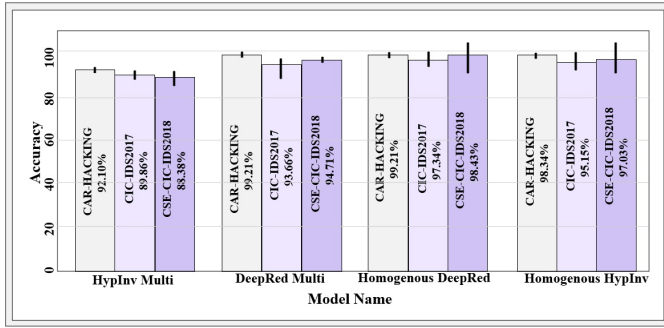


Fig. 12. Accuracy comparison between multi-class and two stage class classification for both approaches DeepRed, and HypInv.

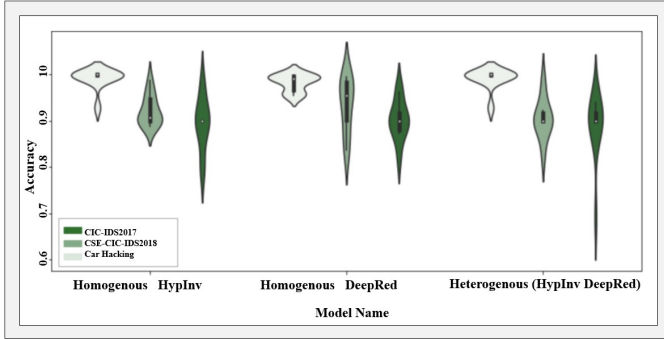


Fig. 13. Visualization of Two-stage variants comparison, the Homogeneous and the heterogeneous on accuracy basis through violins as a density plots to show the peaks in the data.

is performed in Table VI. Similarly, Figure 13 demonstrates the classification performance in the form of violin plots indicating the superior performance of heterogeneous models. The homogeneous model, which is constructed using *HypInv* in two stages, provides the lowest classification performance among the three variants. We can also observe that Homogeneous *DeepRed* is better than homogeneous *HypInv* and heterogeneous variants.

The two-stage classification technique helps in decreasing the complexity of computational of an algorithm [44] in both stages. As a result, the performance of the intrusion detection system increases. The proposed two-stage homogeneous *DeepRed* can extract rules that have accurate estimation when the size of the dataset increases. [13]. *DeepRed* actually needs a reasonable amount of data to extract acceptable rules, as it depends on the neural network internal structure. Thus, compared to the pedagogical approach, *DeepRed* as decompositional has more efficient representation for the DNN. *HypInv* as pedagogical approach generates a decision tree with relatively accurate oblique splits. However, these

splits are not comprehensible as much as *DeepRed* because *HypInv* contains the weighted sum of all attributes.

VII. CONCLUSION

The objectives of this work are two-fold. The first objective is to ensure that the proposed intrusion detection approach can be applicable with less resource consumption in terms of time, CPU, and memory usage in ITS environment. Moreover, the extensive experimental results successfully showed that rule extraction outperforms deep learning in that manner. The second objective is to extract explainable rules from deep neural network by designing two stages of rule extraction, which combines *DeepRed* and *HypInv*, to increase effectiveness detection for large scale datasets. Experimental results show that DNNs are resource hungry models as compared to models constructed through rule extraction regardless of the used rule extraction approach, i.e., either pedagogical or decompositional. By analyzing the results of rule extraction methods, we can conclude that *DeepRed* consumes less resource as compared to *HypInv* and therefore the use of rule extraction in terms of resource consumption is justified. Also, the results and discussion indicate that the two-stage homogeneous variant utilizing *DeepRed* for binary classification and multiclass classification incurs faster predictions and better classification accuracy. It also outperforms the multi-class IDS model using rule extraction. Despite the advantages of rule extraction methods from DNN, it has been shown that they do not perform well when they deal with very deep neural networks or process a big training dataset [53]. Thus, it is obvious that this limitation has more impact when two-stage classifier is considered.

REFERENCES

- [1] A. Verma, R. Saha, G. Kumar, and T.-H. Kim, "The security perspectives of vehicular networks: A taxonomical analysis of attacks and solutions," *Appl. Sci.*, vol. 11, no. 10, p. 4682, May 2021.
- [2] W. Wu *et al.*, "A survey of intrusion detection for in-vehicle networks," *IEEE Trans. Intell. Transp. Syst.*, vol. 21, no. 3, pp. 919–933, Mar. 2019.
- [3] L. Yang, A. Moubayed, I. Hamieh, and A. Shami, "Tree-based intelligent intrusion detection system in internet of vehicles," in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, Dec. 2019, pp. 1–6.
- [4] J. Ashraf, A. D. Bakhshi, N. Moustafa, H. Khurshid, A. Javed, and A. Beheshti, "Novel deep learning-enabled LSTM autoencoder architecture for discovering anomalous events from intelligent transportation systems," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 7, pp. 4507–4518, Jul. 2021.
- [5] M.-J. Kang and J.-W. Kang, "Intrusion detection system using deep neural network for in-vehicle network security," *PLoS ONE*, vol. 11, no. 6, Jun. 2016, Art. no. e0155781.

- [6] L. Nie, Z. Ning, X. Wang, X. Hu, J. Cheng, and Y. Li, "Data-driven intrusion detection for intelligent internet of vehicles: A deep convolutional neural network-based method," *IEEE Trans. Netw. Sci. Eng.*, vol. 7, no. 4, pp. 2219–2230, Oct. 2020.
- [7] H. M. Song, J. Woo, and H. K. Kim, "In-vehicle network intrusion detection using deep convolutional neural network," *Veh. Commun.*, vol. 21, Jan. 2020, Art. no. 100198.
- [8] A. Aldweesh, A. Derhab, and A. Z. Emam, "Deep learning approaches for anomaly-based intrusion detection systems: A survey, taxonomy, and open issues," *Knowl.-Based Syst.*, vol. 189, Feb. 2020, Art. no. 105124.
- [9] T. Hailesilassie, "Rule extraction algorithm for deep neural networks: A review," 2016, *arXiv:1610.05267*.
- [10] J. Townsend, T. Chaton, and J. M. Monteiro, "Extracting relational explanations from deep neural networks: A survey from a neural-symbolic perspective," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 31, no. 9, pp. 3456–3470, Sep. 2019.
- [11] A. Derhab, M. Belaoued, I. Mohiuddin, F. Kurniawan, and M. K. Khan, "Histogram-based intrusion detection and filtering framework for secure and safe in-vehicle networks," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 3, pp. 2366–2379, Mar. 2022.
- [12] G. Bologna and Y. Hayashi, "A comparison study on rule extraction from neural network ensembles, boosted shallow trees, and SVMs," *Appl. Comput. Intell. Soft Comput.*, vol. 2018, pp. 1–20, Jan. 2018.
- [13] J. R. Zilke, E. L. Mencía, and F. Janssen, "DeepRED—Rule extraction from deep neural networks," in *Proc. Int. Conf. Discovery Sci.* Cham, Switzerland: Springer, 2016, pp. 457–473.
- [14] E. W. Saad and D. C. Wunsch, "Neural network explanation using inversion," *Neural Netw.*, vol. 20, no. 1, pp. 78–93, 2007.
- [15] W. Zong, Y.-W. Chow, and W. Susilo, "A two-stage classifier approach for network intrusion detection," in *Proc. Int. Conf. Inf. Secur. Pract. Exper.* Cham, Switzerland: Springer, 2018, pp. 329–340.
- [16] F. A. Khan, A. Gumaei, A. Derhab, and A. Hussain, "A novel two-stage deep learning model for efficient network intrusion detection," *IEEE Access*, vol. 7, pp. 30373–30385, 2019.
- [17] G. Loukas, E. Karapistoli, E. Panaousis, P. Sarigiannidis, and T. Vuong, "A taxonomy and survey of cyber-physical intrusion detection approaches for vehicles," *Ad Hoc Netw.*, vol. 84, pp. 124–147, Mar. 2019.
- [18] C. Young, J. Zambreno, H. Olufowobi, and G. Bloom, "Survey of automotive controller area network intrusion detection systems," *IEEE Des. Test. Comput.*, vol. 36, no. 6, pp. 48–55, Dec. 2019.
- [19] H. Ji, Y. Wang, H. Qin, Y. Wang, and H. Li, "Comparative performance evaluation of intrusion detection methods for in-vehicle networks," *IEEE Access*, vol. 6, pp. 37523–37532, 2018.
- [20] E. Aliwa, O. Rana, C. Perera, and P. Burnap, "Cyberattacks and countermeasures for in-vehicle networks," *ACM Comput. Surveys*, vol. 54, no. 1, pp. 1–37, Apr. 2021.
- [21] O. Y. Al-Jarrah, C. Maple, M. Dianati, D. Oxtoby, and A. Mouzakitis, "Intrusion detection systems for intra-vehicle networks: A review," *IEEE Access*, vol. 7, pp. 21266–21289, 2019.
- [22] Z.-S. Pan, S.-C. Chen, G.-B. Hu, and D.-Q. Zhang, "Hybrid neural network and C4.5 for misuse detection," in *Proc. Int. Conf. Mach. Learn. Cybern.*, vol. 4, 2003, pp. 2463–2467.
- [23] A. Hofmann, C. Schmitz, and B. Sick, "Rule extraction from neural networks for intrusion detection in computer networks," in *Proc. Conf. IEEE Int. Conf. Syst., Man Cybernetics. Conf. Theme-Syst. Secur. Assurance*, vol. 2, Oct. 2003, pp. 1259–1265.
- [24] F. Safara, A. Souri, and M. Serrizadeh, "Improved intrusion detection method for communication networks using association rule mining and artificial neural networks," *IET Commun.*, vol. 14, no. 7, pp. 1192–1197, Apr. 2020.
- [25] S. Mahdaviifar and A. A. Ghorbani, "DeNNes: Deep embedded neural network expert system for detecting cyber attacks," *Neural Comput. Appl.*, vol. 32, no. 18, pp. 14753–14780, Sep. 2020.
- [26] A. Yan *et al.*, "Effective detection of mobile malware behavior based on explainable deep neural network," *Neurocomputing*, vol. 453, pp. 482–492, Sep. 2021.
- [27] A. Shiravi, H. Shiravi, M. Tavallaei, and A. A. Ghorbani, "Toward developing a systematic approach to generate benchmark datasets for intrusion detection," *Comput. Secur.*, vol. 31, no. 3, pp. 357–374, 2012.
- [28] R. Panigrahi and S. Borah, "A detailed analysis of CICIDS2017 dataset for designing intrusion detection systems," *Int. J. Eng. Technol.*, vol. 7, pp. 479–482, Dec. 2018.
- [29] J. L. Leevy and T. M. Khoshgoftaar, "A survey and analysis of intrusion detection models based on CSE-CIC-IDS2018 big data," *J. Big Data*, vol. 7, no. 1, pp. 1–19, Dec. 2020.
- [30] E. Seo, H. M. Song, and H. K. Kim, "GIDS: GAN based intrusion detection system for in-vehicle network," in *Proc. 16th Annu. Conf. Privacy, Secur. Trust (PST)*, Aug. 2018, pp. 1–6.
- [31] C. Aldrich, "Extraction of rules from data with neural networks," in *Exploratory Analysis of Metallurgical Process Data With Neural Networks and Related Methods* (Process Metallurgy), vol. 12, C. Aldrich, Ed. Amsterdam, The Netherlands: Elsevier, 2002, ch. 7, pp. 228–261. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1572440902800086>
- [32] T. A. Etchells and P. J. G. Lisboa, "Orthogonal search-based rule extraction (OSRE) for trained neural networks: A practical and efficient approach," *IEEE Trans. Neural Netw.*, vol. 17, no. 2, pp. 374–384, Mar. 2006.
- [33] R. Setiono, W. K. Leow, and J. M. Zurada, "Extraction of rules from artificial neural networks for nonlinear regression," *IEEE Trans. Neural Netw.*, vol. 13, no. 3, pp. 564–577, May 2002.
- [34] L. Özbakır, A. Baykasoglu, and S. Kulluk, "A soft computing-based approach for integrated training and rule extraction from artificial neural networks: DIFACONN-miner," *Appl. Soft Comput.*, vol. 10, no. 1, pp. 304–317, Jan. 2010.
- [35] M. Sato and H. Tsukimoto, "Rule extraction from neural networks via decision tree induction," in *Proc. Int. Joint Conf. Neural Netw.*, vol. 3, 2001, pp. 1870–1875.
- [36] M. Chakraborty, S. K. Biswas, and B. Purkayastha, "Rule extraction from neural network trained using deep belief network and back propagation," *Knowl. Inf. Syst.*, vol. 62, no. 9, pp. 3753–3781, Sep. 2020, doi: [10.1007/s10115-020-01473-0](https://doi.org/10.1007/s10115-020-01473-0).
- [37] M. G. Augasta and T. Kathirvalavakumar, "Reverse engineering the neural networks for rule extraction in classification problems," *Neural Process. Lett.*, vol. 35, no. 2, pp. 131–150, Apr. 2012.
- [38] R. Setiono, B. Baesens, and C. Mues, "Recursive neural network rule extraction for data with mixed attributes," *IEEE Trans. Neural Netw.*, vol. 19, no. 2, pp. 299–307, Feb. 2008.
- [39] S. Odense and A. d'Avila Garcez, "Layerwise knowledge extraction from deep convolutional networks," 2020, *arXiv:2003.09000*.
- [40] G. Bologna, "A simple convolutional neural network with rule extraction," *Appl. Sci.*, vol. 9, no. 12, p. 2411, Jun. 2019. [Online]. Available: <https://www.mdpi.com/2076-3417/9/12/2411>
- [41] G. Bovenzi, G. Aceto, D. Ciunzio, V. Persico, and A. Pescapé, "A hierarchical hybrid intrusion detection approach in IoT scenarios," in *Proc. IEEE Global Commun. Conf.*, Dec. 2020, pp. 1–7.
- [42] N. Koroniotis, N. Moustafa, E. Sitnikova, and B. Turnbull, "Towards the development of realistic botnet dataset in the Internet of Things for network forensic analytics: Bot-IoT dataset," *Future Gener. Comput. Syst.*, vol. 100, pp. 779–796, Nov. 2019.
- [43] A. Khan and S. Khan, "Two level anomaly detection classifier," in *Proc. Int. Conf. Comput. Electr. Eng.*, Dec. 2008, pp. 65–69.
- [44] J. Hussain, S. Lalmuanawma, and L. Chhakchhuak, "A two-stage hybrid classification technique for network intrusion detection system," *Int. J. Comput. Intell. Syst.*, vol. 9, no. 5, pp. 863–875, 2016.
- [45] M. F. Umer, M. Sher, and Y. Bi, "A two-stage flow-based intrusion detection model for next-generation networks," *PLoS ONE*, vol. 13, no. 1, Jan. 2018, Art. no. e0180945.
- [46] L. Li, Y. Yu, S. Bai, Y. Hou, and X. Chen, "An effective two-step intrusion detection approach based on binary classification and k -NN," *IEEE Access*, vol. 6, pp. 12060–12073, 2018.
- [47] M. Espinosa Zarlenga, Z. Shams, and M. Jamnik, "Efficient decompositional rule extraction for deep neural networks," 2021, *arXiv:2111.12628*.
- [48] X. Glorot, A. Bordes, and Y. Bengio, "Deep sparse rectifier neural networks," in *Proc. 14th Int. Conf. Artif. Intell. Statist.*, 2011, pp. 315–323.
- [49] M. Fanta, P. Pulc, and M. Holena, "Rules extraction from neural networks trained on multimedia data," in *Proc. ITAT*, 2019, pp. 26–35.
- [50] L. F. Kozachenko and N. N. Leonenko, "Sample estimate of the entropy of a random vector," *Problemy Peredachi Inform.*, vol. 23, no. 2, pp. 9–16, Oct. 1987.
- [51] A. E. Khedr, A. M. Idrees, and A. I. El Seddawy, "Enhancing iterative Dichotomiser 3 algorithm for classification decision tree," *Wiley Interdiscipl. Reviews, Data Mining Knowl. Discovery*, vol. 6, no. 2, pp. 70–79, Mar. 2016.
- [52] Google Cloud: Cloud Computing Services. Accessed: Jun. 14, 2022. [Online]. Available: <https://cloud.google.com>
- [53] M. Espinosa Zarlenga, Z. Shams, and M. Jamnik, "Efficient decompositional rule extraction for deep neural networks," 2021, *arXiv:2111.12628*.

Samah Almutlaq received the B.Sc. and M.Sc. degrees in information systems from the College of Computer and Information Sciences, King Saud University, Riyadh, Saudi Arabia, in 2017 and 2019, respectively. She is currently pursuing the Ph.D. degree with the Information Systems Department, College of Computer and Information Sciences, King Saud University, Riyadh, Saudi Arabia. Her main research interests include cyber security, networks, and machine learning.

Abdelouahid Derhab received the engineer's, M.Sc., and Ph.D. degrees in computer science from the University of Sciences and Technology Houari Boumediene (USTHB), Algiers, in 2001, 2003, and 2007, respectively. He is currently an Associate Professor with the Center of Excellence in Information Assurance (COEIA), King Saud University. He is the author of more than 110 papers in different peer-reviewed journals and conferences. His research interests include malware analysis, network security, intrusion detection, mobile security, the Internet of Things, smart grid, blockchain, and cyber security policies. He was a workshop chair, a technical committee chair, and a reviewer for many journals and international conferences.

Mohammad Mehedi Hassan (Senior Member, IEEE) received the Ph.D. degree in computer engineering from Kyung Hee University, Seoul, South Korea, in February 2011. He is currently a Full Professor with the Information Systems Department, College of Computer and Information Sciences, King Saud University, Riyadh, Saudi Arabia. He has authored and coauthored around over 300 publications, including refereed IEEE/ACM/Springer/Elsevier journals, conference papers, books, and book chapters. His research interests include edge/cloud computing, the Internet of Things, cyber security, deep learning, artificial intelligence, body sensor networks, 5G networks, and social networks.

Kuljeet Kaur (Member, IEEE) received her Ph.D. degree from the Thapar Institute of Engineering and Technology in 2018. She was a NSERC Post-Doctoral Research Fellow at the ÉTS, Montréal, Canada, from 2018 to 2020. She is currently working as an Assistant Professor with the Electrical Engineering Department, École de Technologie Supérieure (ÉTS), Université du Québec, Montreal and a Visiting Researcher with the School of Computer Science and Engineering (SCSE), Nanyang Technological University (NTU), Singapore. She has over 55 publications in highly ranked journals and conferences. Her main research interests include cloud computing, energy efficiency, smart grid, frequency support, and vehicle-to-grid. She is a Website Co-Chair of the N2Women Community. She is the Vice-Chair of the IEEE Montreal Young Professionals Affinity Group.